

fix-hostfiles

hblock(1) blocks advertising, tracking, and malware domains by mapping them to 0.0.0.0 in `/etc/hosts`. Occasionally a domain that should remain reachable is included. **fix-hostfiles** manages the backup/restore workflow and adds those domains to hblock's allow list.

Requirements

- A C11 compiler
- hblock for the `prep` action
- macOS for DNS-cache flushing
- Sufficient privileges for changes beneath `/etc`

Build and test

```
make
make test
make release
```

The default build enables strict compiler warnings. `make test` also runs nine isolated behavior tests. The tests redirect the hosts and hblock paths to a temporary fixture and never modify `/etc`.

Usage

```
fix-hostfiles [OPTIONS] <ACTION>
```

Actions:

<code>prep</code>	Back up <code>/etc/hosts</code> and run hblock
<code>restore</code>	Restore <code>/etc/hosts</code> from <code>/etc/hosts-ORIG</code>

Options that act as actions:

<code>-a, --add DNS_NAME</code>	Add a domain to the allow list and unblock it
<code>-f, --flush</code>	Flush the macOS DNS cache and restart mDNSResponder
<code>-h, --help</code>	Display help

Modifier:

<code>-v, --verbose</code>	Display detailed paths, commands, and file metadata
----------------------------	---

Only one action may be supplied at a time. System-file operations should normally be run with `sudo`:

```
sudo fix-hostfiles prep
sudo fix-hostfiles restore
sudo fix-hostfiles --add example.domain.com
sudo fix-hostfiles --flush
sudo fix-hostfiles --verbose prep
```

Both `prep` and `restore` ask before overwriting an existing destination file. The add operation creates `/etc/hosts.bak`, adds the DNS name only when absent, and removes only a whitespace-delimited exact match from `/etc/hosts`.

Host-file removal uses atomic replacement: the complete revised contents are written to a uniquely named temporary file beside the active hosts file, flushed to disk, and assigned the original permissions and ownership. A same-filesystem `rename(2)` then replaces the active file in one operation, so readers see either the complete old version or the complete new version rather than a partially rewritten file. If preparation fails before the rename, the original remains in place and the temporary file is removed.

Verbose mode reports the relevant file type, symbolic and octal permissions, owner, group, size, inode, link count, and modification time before and after a filesystem action. For DNS flushing, it reports the platform and commands.

Test configuration

`FIX_HOSTFILES_ETC_DIR` and `FIX_HOSTFILES_HBLOCK_DIR` override the production paths. They exist to support isolated tests and development fixtures.

Install the executable and man page under `/opt/homebrew` with `make install`.

Documentation

- **Manual page:** `man ./fix-hostfiles.1` (hand-written roff; check with `mandoc -Tlint fix-hostfiles.1`). `make install` puts it in a `man1` folder under `/opt/homebrew`.
- **Algebraic specification:** the four actions stated as transformations of the hosts file and allow list, with twelve laws (removal and insertion idempotence, commutativity, the prep-restore round trip, verbose noninterference, and others), how atomic replacement refines them, and the property tests they suggest.
- **API documentation:** every function, type and macro, generated by Doxygen from the source comments; the HTML version starts at `docs/html/index.html`.
- **Presentation:** thirteen Typst slides on how `fix-hostfiles` is built: the architecture, argument parsing, each action, atomic replacement and the laws, with a call graph generated from the code by clang. `make -C docs/talk` rebuilds it (needs Typst, clang, Graphviz and Python 3).
- **README as PDF:** this file, rendered by Pandoc.

The built PDFs are committed next to their sources. To rebuild them:

```
make -C algebra    # algebraic specification (needs latexmk)
make docs          # readme.pdf and the API docs (needs Pandoc, Doxygen, LaTeX)
make -C docs/talk  # presentation
```

The companion Bash implementation lives in the separate `fix-hosts-bash` repository.